C++ COMPLETE REFERENCE LIBRARY • MODULE 04

04 CONTROL FLOW — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 conditional if else and switch
3. **Theory Chapter:** 02 loops for while do while and range for
4. **Theory Chapter:** 03 jump statements and common pitfalls
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

04 — Control Flow

This module covers selection statements (`if` , `if-else` , `switch-case`), iteration statements (`for` , range-based `for` , `while` , `do-while`), and jump statements (`break` , `continue` , `return`).

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_conditional_if_else_and_switch.md` `(./01_THEORY/01_conditional_if_else_and_switch.md)`
  - `02_loops_for_while_do_while_and_range_for.md` `(./01_THEORY/02_loops_for_while_do_while_and_range_for.md)`
  - `03_jump_statements_and_common_pitfalls.md` `(./01_THEORY/03_jump_statements_and_common_pitfalls.md)`
- **Programs (`02_PROGRAMS/`)**:
  - `01_if_else_and_switch_grade.cpp` `(./02_PROGRAMS/01_if_else_and_switch_grade.cpp)` (Category A)
  - `02_loops_prime_and_fibonacci.cpp` `(./02_PROGRAMS/02_loops_prime_and_fibonacci.cpp)` (Category A)

  - `03_switch_float_compile_error.cpp` (./02_PROGRAMS/03_switch_float_compile_error.cpp) (Category B)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_loop_control_flow.svg` (./05_DIAGRAMS/diagram_01_loop_control_flow.svg)

01 — Conditional Selection: `if-else` and `switch-case`

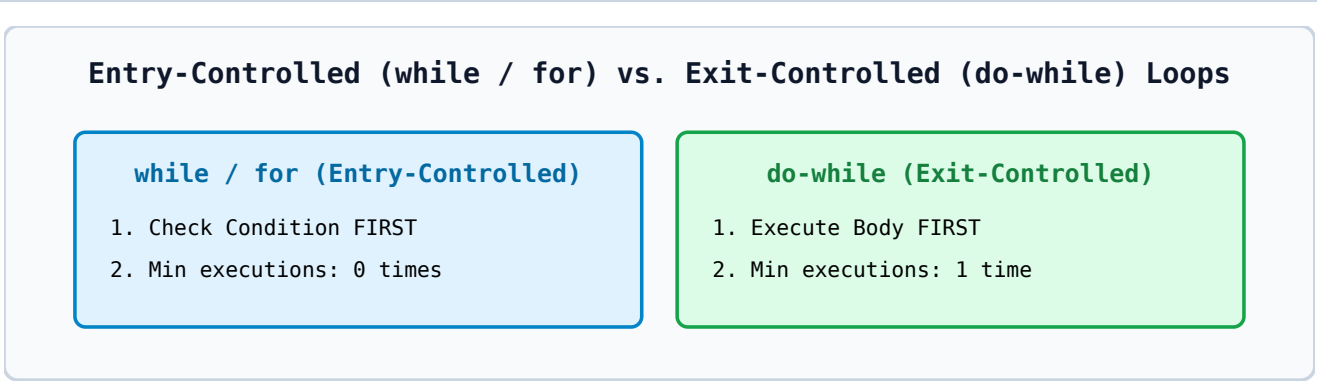
1. `if-else` Pitfall: Assignment `=` vs. Equality `==`

Writing `if (x = 5)` assigns `5` to `x` and evaluates to `true` (because `5 ≠ 0`)! Always use `==` for comparison.

2. `switch` Statement Rules (`[stmt.switch]`)

- The condition of a `switch(expr)` **must** have an **integral or enumeration type** (`int` , `char` , `enum`). Floating-point types (`float` , `double`) and `std::string` are **Compilation Errors** in `switch` !
- Every `case` label must be an integral constant expression (`constexpr`).
- Omitting `break;` causes execution to **fall through** into the next `case` block!

02 — Loops: `for`, `while`, `do-while`, and Range-Based `for`



Loop Control Flow

1. `while` **Loop (Entry-Controlled)**: Evaluates condition **before** executing the body (`0` or more iterations).
2. `do-while` **Loop (Exit-Controlled)**: Executes the body **at least once** before checking the condition at the bottom (`do { ... } while (cond);`).
3. `for` **Loop**: `for (init; condition; update) { ... }`.
4. **Range-Based** `for` **Loop (C++11–C++23)**: `for (int x : arr)` or `for (const auto& elem : container)`.

03 — Jump Statements (`break`, `continue`, `return`)

- `break`: Immediately terminates the innermost enclosing loop or `switch` statement.
- `continue`: Skips the remaining statements of the current loop iteration and jumps to the loop's next iteration check (in a `for` loop, `continue` jumps to the `update` expression first!).

Visual Concept & Memory Diagrams (04 CONTROL FLOW)

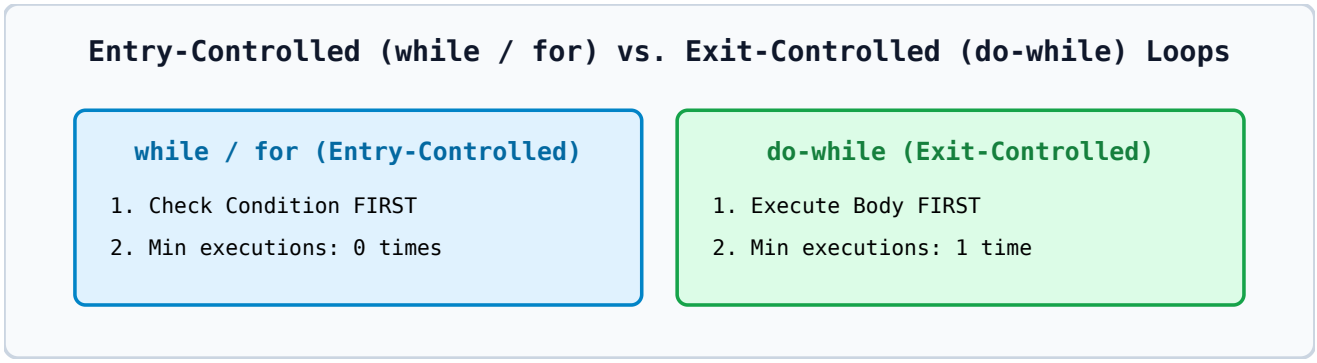


Diagram: diagram 01 loop control flow

Annotated C++ Programs (04 CONTROL FLOW)

Program: `01_if_else_and_switch_grade.cpp`

C++ SOURCE FILE • 01_if_else_and_switch_grade.cpp

```cpp
// Program Title: Grade Classification Using if-else and switch-case
// Topic: Control Flow — Selection Statements
// Purpose: Demonstrate if-else ladder and switch-case with break statements.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_conditional_if_else_and_switch.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int marks = 85;
    char grade;

    if (marks ⩾ 90) {
        grade = 'A';
    } else if (marks ⩾ 80) {
        grade = 'B';
    } else {
        grade = 'C';
    }

    switch (grade) {
        case 'A':
            cout << "Grade A: Excellent" << endl;
            break;
        case 'B':
            cout << "Grade B: Very Good" << endl;
            break;
        default:
            cout << "Grade C: Good" << endl;
            break;
    }

    return 0;
}

/*
Expected Deterministic Output:
Grade B: Very Good
*/
```

Program: `02_loops_prime_and_fibonacci.cpp`

C++ SOURCE FILE • 02_loops_prime_and_fibonacci.cpp

```cpp
// Program Title: Prime Check and Fibonacci Sequence Using Loops
// Topic: Control Flow — Iteration Statements
// Purpose: Demonstrate for and while loops on standard B.Tech exam problems.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_loops_for_while_do_while_and_range_for.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int n = 17;
    bool is_prime = true;

    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            is_prime = false;
            break;
        }
    }

    cout << n << " is prime: " << is_prime << endl;

    cout << "First 6 Fibonacci numbers: ";
    int a = 0, b = 1;
    for (int k = 0; k < 6; k++) {
        cout << a << " ";
        int next = a + b;
        a = b;
        b = next;
    }
    cout << endl;

    return 0;
}

/*
Expected Deterministic Output:
17 is prime: 1
First 6 Fibonacci numbers: 0 1 1 2 3 5
*/
```

Program: `03_switch_float_compile_error.cpp`

C++ SOURCE FILE • 03_switch_float_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Non-Integral Condition in switch Statement
// Topic: Control Flow — switch Type Restrictions
// Purpose: Show why passing a double or float to switch(x) fails to compile.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/01_conditional_if_else_and_switch.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

int main() {
    double val = 1.5;
    switch (val) { // COMPILE ERROR: switch quantity not an integer
        case 1:
            cout << "One" << endl;
            break;
    }
    return 0;
}

/*
Why Compilation Fails:
ISO C++ [stmt.switch] requires the switch condition to be of integral or enumeration type.
Use an 'if-else' statement when branching on floating-point values.
*/
```

Conceptual Questions — Module 04: Control Flow

Q1. Compare `while` and `do-while` loops. **Q2.** What types are allowed inside `switch(expression)` in C++?

Debugging Questions — Module 04

Q4. Find the infinite loop bug in `for (int i = 0; i < 5; i++); { cout << i; }`.

Predict the Output Questions — Module 04

Q3. Predict the output when `break;` is omitted in a `switch` statement (fallthrough).

Programming Questions — Module 04

Q5. Write a C++23 program to reverse the digits of an integer and check if it is a palindrome.

Detailed Solutions — Conceptual Questions (Module 04)

- **Q1 Solution**: `while` checks the condition before entering the body (executes `0` or more times); `do-while` checks the condition after executing the body (executes at least `1` time).
- **Q2 Solution**: Only integral types (`int`, `char`, `short`, `long`, `long long`, `bool`) and enumeration types (`enum`, `enum class`).

Detailed Solutions — Predict the Output & Debugging (Module 04)

- **Q3 Solution**: Without `break;`, execution falls through and runs all subsequent `case` blocks until a `break;` or the end of the `switch` block is reached.
- **Q4 Solution**: The stray semicolon `;` right after `for (...)` terminates the loop with an empty body, and `i` is out of scope in `{ cout << i; }`.

Detailed Solutions — Programming Questions (Module 04)

CPP

```cpp
#include <iostream>
using namespace std;
int main() {
    int n = 1221, original = 1221, rev = 0;
    while (n > 0) {
        rev = (rev * 10) + (n % 10);
        n = n / 10;
    }
    cout << "Reversed: " << rev << ", Palindrome: " << (original == rev) << endl;
    return 0;
}
```